LofiSoft — Documentation de l'API

Oct 28, 2026

Contrat entre l'application Flutter et l'API Laravel : conventions, routes par module, formats d'entrée et de sortie. C'est une proposition à valider par l'équipe avant de coder le back.

Conventions générales

Toutes les routes sont en JSON, sous le préfixe `/api`, et celles qui ne sont pas publiques demandent un jeton.

Sujet	Règle
URL de base	`http://10.0.2.2:8000/api` en local (émulateur Android), valeur réglable dans l'app par `API_BASE_URL`
Format	Requêtes et réponses en `application/json`, texte en UTF-8
Authentification	Jeton Bearer (Laravel Sanctum) : `Authorization: Bearer <token>`. Le jeton est renvoyé par `/auth/login` et `/auth/register`
Identifiants	Entiers ou UUID en texte, toujours lus comme du texte par l'app
Montants	Entiers en FCFA, sans décimale (`75000`), jamais de texte formaté
Dates	ISO 8601 en UTC (`2026-10-28T12:30:00Z`) ; les échéances sont des dates seules (`2026-11-05`)
Téléphone	10 chiffres commençant par `01`, envoyés sans `+229` (à confirmer)
Pagination	`?page=1&per_page=20`, réponse avec `data` et `meta` (`current_page`, `last_page`, `total`)
Délai	L'app abandonne après 20 secondes

Succès. Code 200 (lecture), 201 (création), 204 (suppression sans contenu). Les objets sont dans `data` ; l'app accepte aussi `token` et `user` à la racine pour la connexion.

Erreurs. Le corps est toujours `{ "message": "...", "errors": { "champ": ["texte"] } }` ; `errors` n'existe que pour le code 422. Le texte de `message` est montré tel quel à l'utilisateur, donc il est écrit en français.

Code	Quand	Comportement de l'app
400 / 422	Données invalides	Affiche la première erreur de `errors`
401	Jeton absent ou expiré ; ou identifiants faux sur `/auth/login`	Vide la session et revient à la connexion
403	Rôle non autorisé pour cette route	Affiche `message`
404	Ressource introuvable	Affiche `message`
409	Conflit (e-mail déjà pris, logement déjà rejoint)	Affiche `message`
5xx	Erreur serveur	Message générique, bouton Réessayer

Authentification et profil

Les quatre premières routes existent déjà côté Flutter (`AuthService`) : le back doit respecter exactement ces noms de champs.

Méthode	Route	Accès	Rôle
POST	`/auth/register`	Public	Crée un compte et renvoie la session
POST	`/auth/login`	Public	Ouvre une session
GET	`/auth/me`	Jeton	Renvoie l'utilisateur connecté
POST	`/auth/forgot-password`	Public	Envoie un lien de réinitialisation par e-mail
POST	`/auth/logout`	Jeton	Révoque le jeton courant
PUT	`/me`	Jeton	Modifie nom, prénoms, téléphone
PUT	`/me/password`	Jeton	Change le mot de passe (`current_password` , `password`)

POST /auth/register

```
{
  "nom": "Mensah",
  "prenoms": "Koffi Patrick",
  "email": "koffi@exemple.bj",
  "telephone": "0197000000",
  "password": "********",
  "role": "TENANT"
}
```

`role` vaut `TENANT` ou `OWNER` (jamais `ADMIN` à l'inscription). L'e-mail et le téléphone sont uniques : sinon 422 avec `errors.email` ou `errors.telephone`. Réponse 201 :

```
{
  "token": "1|abc...",
  "user": {
    "id": "12", "nom": "Mensah", "prenoms": "Koffi Patrick",
    "email": "koffi@exemple.bj", "telephone": "0197000000",
    "role": "TENANT", "actif": true
  }
}
```

POST /auth/login : corps `{ "email", "password" }`, même réponse que l'inscription. Mauvais identifiants : 401 avec `message: "E-mail ou mot de passe incorrect."`. Un compte `actif: false` reçoit 403.

GET /auth/me : renvoie `{ "user": { ... } }` avec les mêmes champs. L'app s'en sert au lancement pour savoir vers quel espace envoyer la personne (locataire, propriétaire, admin).

POST /auth/forgot-password : corps `{ "email" }`. Répond toujours 200 avec le même message, que l'e-mail existe ou non, pour ne pas révéler les comptes.

Annonces et favoris

Le fil est public (lecture sans compte) ; les favoris demandent un jeton locataire. Une annonce est publiée par un propriétaire dont l'identité est vérifiée.

Méthode	Route	Accès	Rôle
GET	`/announcements`	Public	Fil des annonces `PUBLISHED`, filtres et pagination
GET	`/announcements/{id}`	Public	Détail d'une annonce

Méthode	Route	Accès	Rôle
POST	`/announcements`	Propriétaire	Crée une annonce (brouillon `DRAFT`)
PUT	`/announcements/{id}`	Propriétaire	Modifie sa propre annonce
POST	`/announcements/{id}/publish`	Propriétaire	Passe l'annonce en `PUBLISHED`
POST	`/announcements/{id}/cancel`	Propriétaire	Passe l'annonce en `CANCELED`
GET	`/me/announcements`	Propriétaire	Ses annonces, tous statuts
GET	`/me/favorites`	Locataire	Annonces mises en favori
PUT	`/announcements/{id}/favorite`	Locataire	Ajoute en favori (sans effet si déjà fait)
DELETE	`/announcements/{id}/favorite`	Locataire	Retire des favoris
POST	`/announcements/{id}/reports`	Jeton	Signale une annonce (`reason`, `details`)

GET /announcements accepte :

Paramètre	Exemple	Sens
`city`	`Cotonou`	Ville
`type`	`HOUSE`	Type de logement
`min_price`, `max_price`	`30000`, `80000`	Fourchette de loyer mensuel (FCFA)
`q`	`calavi`	Recherche libre dans titre et quartier
`page`, `per_page`	`1`, `20`	Pagination

Objet annonce (fil et détail ; le détail ajoute `description` et `owner`) :

```
{
  "id": "45",
  "title": "Studio meublé à Fidjrossè",
  "type": "ROOM",
  "rent": 75000,
  "deposit_months": 2,
  "city": "Cotonou",
  "district": "Fidjrossè",
  "rooms": 1,
  "photos": ["https://res.cloudinary.com/<cloud>/image/upload/v1/lofisoft/45-
1.jpg"],
  "verified": true,
  "is_favorite": false,
  "status": "PUBLISHED",
  "published_at": "2026-10-20T09:00:00Z",
  "description": "...",
  "owner": { "id": "7", "name": "Koffi Mensah" }
}
```

`verified` indique que le propriétaire a passé la vérification d'identité (c'est l'étiquette « Vérifié » de la carte d'annonce). `is_favorite` n'est rempli que si un jeton est envoyé, sinon il vaut `false`.

Logements, locations et code logement

Un propriétaire enregistre un logement et reçoit un code unique (`M4T-83P`, 6 caractères avec tiret). Le locataire saisit ce code pour rejoindre le logement : c'est l'écran « Rejoindre mon logement ». Le lien entre un locataire et un logement est une location (`rental`).

Méthode	Route	Accès	Rôle
POST	`/housings`	Propriétaire	Crée un logement, renvoie son `code`
GET	`/housings`	Propriétaire	Ses logements
GET	`/housings/{id}`	Propriétaire	Détail d'un de ses logements
PUT	`/housings/{id}`	Propriétaire	Modifie loyer, échéance, adresse
GET	`/housings/lookup?code=M4T-83P`	Locataire	Montre le logement correspondant au code, avant de confirmer
POST	`/rentals/join`	Locataire	Rejoint le logement avec son code

Méthode	Route	Accès	Rôle
GET	`/me/rental`	Locataire	La location en cours du locataire
POST	`/rentals/{id}/end`	Propriétaire	Termine la location (`FINISHED`)

POST /housings :

```
{
  "title": "Appartement 3C",
  "type": "HOUSE",
  "rent": 85000,
  "due_day": 5,
  "city": "Cotonou",
  "district": "Akpakpa",
  "photos": ["https://res.cloudinary.com/..."]
}
```

`due_day` est le jour du mois avant lequel le loyer doit être payé (de 1 à 28). La réponse contient le logement et son `code`.

GET /housings/lookup renvoie `{ "housing": { "id", "title", "city", "district", "rent", "owner_name" } }`. Code inconnu : 404 avec `message: "Ce code ne correspond à aucun logement."`.

POST /rentals/join : corps `{ "code": "M4T-83P" }`. La location est créée en `PENDING` et le propriétaire la confirme depuis son espace (voir Espace propriétaire). Un locataire ne peut avoir qu'une location `ACTIVE` ou `PENDING` à la fois : sinon 409.

Loyers et paiements

Chaque mois d'une location a une ligne de loyer (`rent_period`) avec ce qui est dû et ce qui est déjà payé : un paiement peut être partiel. Les paiements se font en Mobile Money (MTN, Moov) ; l'agrégateur de paiement (KKiaPay) est encore à confirmer, donc la route de paiement est volontairement générique.

Méthode	Route	Accès	Rôle
GET	`/me/rent`	Locataire	Loyer du mois courant : dû, payé, reste, échéance, statut
GET	`/me/rent/periods`	Locataire	Les mois précédents (jauge des mois payés, partiels, en retard)

Méthode	Route	Accès	Rôle
POST	`/payments`	Locataire	Démarre un paiement Mobile Money
GET	`/payments/{id}`	Locataire	État d'un paiement (pour savoir s'il a réussi)
GET	`/me/payments`	Locataire	Historique des paiements
GET	`/payments/{id}/receipt`	Locataire	Données du reçu
POST	`/payments/webhook`	Agrégateur	Confirmation de paiement envoyée par le prestataire (signature vérifiée)

GET /me/rent :

```
{
  "period": "2026-11",
  "label": "novembre",
  "total": 150000,
  "paid": 90000,
  "remaining": 60000,
  "due_date": "2026-11-05",
  "status": "PARTIAL"
}
```

`status` vaut `PAID`, `PARTIAL`, `LATE` ou `UPCOMING`. L'app affiche « Payé », « Partiel » ou « En retard » avec la couleur correspondante.

POST /payments :

```
{
  "rental_id": "3",
  "period": "2026-11",
  "amount": 30000,
  "operator": "MTN",
  "phone": "0197000000"
}
```

Le montant doit être supérieur à 0 et ne pas dépasser le reste à payer, sinon 422. Réponse 201 : `{ "payment": { "id", "status": "PENDING", "amount", "operator" } }`. Le statut passe à `SUCCESS` ou `FAILED` quand le webhook arrive ; l'app interroge `GET /payments/{id}` toutes les 3 secondes pendant l'attente.

GET /payments/{id}/receipt :

```
{
  "reference": "RC-2026-1022-01",
  "amount": 30000,
  "paid_at": "2026-10-22T10:14:00Z",
  "operator": "MTN Mobile Money",
  "housing": "Appartement 3C",
  "period": "2026-11"
}
```

Historique : `GET /me/payments` renvoie la liste paginée `{ "id", "amount", "operator", "paid_at", "period", "status" }`, du plus récent au plus ancien.

Espace propriétaire

Le tableau de bord résume l'argent attendu et reçu du mois, puis liste les locataires avec l'état de leur loyer.

Méthode	Route	Accès	Rôle
GET	`/owner/dashboard`	Propriétaire	Chiffres du mois et locataires en retard
GET	`/owner/tenants`	Propriétaire	Locataires de tous ses logements, avec statut du loyer
GET	`/owner/tenants/{rentalId}`	Propriétaire	Fiche d'un locataire et historique de ses paiements
POST	`/owner/rentals/{id}/accept`	Propriétaire	Confirme un locataire qui a rejoint (`PENDING` vers `ACTIVE`)
POST	`/owner/rentals/{id}/reject`	Propriétaire	Refuse la demande
POST	`/owner/rentals/{id}/remind`	Propriétaire	Envoie un rappel de paiement au locataire
POST	`/owner/payments/manual`	Propriétaire	Note un paiement reçu en espèces

GET /owner/dashboard :

```
{
  "month": "2026-11",
  "expected": 850000,
  "received": 510000,
  "late_count": 2,
  "tenants_count": 8,
  "housings_count": 5
}
```

GET /owner/tenants renvoie, pour chaque location :

```
{
  "rental_id": "3",
  "tenant_name": "Éric Bio-Tchané",
  "housing": "Appartement 3C",
  "due": 170000,
  "carried_over": 50000,
  "status": "LATE"
}
```

Filtre `?status=LATE` pour n'obtenir que les retards. `carried_over` est le montant reporté des mois précédents (affiché sous le nom du locataire, « dont 50 000 reportés »).

POST /owner/payments/manual : corps `{ "rental_id", "period", "amount", "note" }`. Le paiement est créé en `SUCCESS` avec `operator: "CASH"`.

Administration

L'administrateur valide l'identité des propriétaires (pièce d'identité et selfie) et traite les signalements d'annonces. Toutes ces routes sont réservées au rôle `ADMIN`.

Méthode	Route	Rôle
POST	`/me/identity`	Propriétaire : envoie `id_image_url` et `selfie_url` pour vérification
GET	`/admin/identity-requests?status=PENDING`	Liste les demandes de vérification
POST	`/admin/identity-requests/{id}/approve`	Valide : le propriétaire devient `verified`
POST	`/admin/identity-requests/{id}/reject`	Refuse (`reason` obligatoire, montrée au propriétaire)
GET	`/admin/reports?status=PENDING`	Liste les signalements

Méthode	Route	Rôle
POST	`/admin/reports/{id}/resolve`	Traite le signalement (`action` : `REMOVE_ANNOUNCEMENT` ou `WARN_OWNER`)
POST	`/admin/reports/{id}/dismiss`	Ignore le signalement
GET	`/admin/users`	Liste des utilisateurs (filtre `role` , `q`)
PUT	`/admin/users/{id}/active`	Active ou désactive un compte (`actif`)

Demande de vérification :

```
{
  "id": "9",
  "owner": { "id": "7", "name": "Koffi Mensah" },
  "id_image_url": "https://res.cloudinary.com/...",
  "selfie_url": "https://res.cloudinary.com/...",
  "status": "PENDING",
  "created_at": "2026-10-27T08:00:00Z"
}
```

Signalement : `{ "id", "reason", "details", "status", "created_at", "announcement": { "id", "title" }, "reports_count" }` . `reports_count` est le nombre de signalements de la même annonce, affiché sur la carte (« 2 signalements »).

Les pièces d'identité et selfies sont des données sensibles : ils ne sont visibles que par l'admin et jamais renvoyés dans les routes publiques.

Images

Cloudinary n'est pas encore configuré : tant qu'il ne l'est pas, l'API peut renvoyer des listes `photos` vides et l'app affiche une illustration. Le principe retenu est que **le téléphone envoie la photo au back, et le back renvoie l'URL** ; la base ne garde que l'URL.

Méthode	Route	Rôle
POST	`/uploads`	Envoie une image (`multipart/form-data` , champs `file` et `kind`)

`kind` vaut `announcement_photo` , `identity_id` ou `identity_selfie` . Réponse 201 : `{ "url": "https://res.cloudinary.com/<cloud>/image/upload/v1/lofisoft/xyz.jpg" }` .

Règles : JPEG ou PNG, 8 Mo au maximum (l'app compresse avant l'envoi, largeur 1600 px). Les pièces d'identité et selfies sont envoyés en mode privé (`type: authenticated`) et jamais en accès public. L'app ajoute elle-même les transformations d'affichage (`f_auto` , `q_auto` , largeur) à l'URL des photos d'annonce.

Énumérations et modèles de données

Les quatre premières énumérations existent déjà dans le code Flutter (`enums.dart`) et gardent exactement ces valeurs, en majuscules et en anglais. Les autres sont proposées.

Énumération	Valeurs	Utilisée pour
`UserRole`	`TENANT` , `OWNER` , `ADMIN`	Rôle d'un compte
`RentalStatus`	`PENDING` , `ACTIVE` , `FINISHED`	Location d'un locataire
`AnnouncementStatus`	`DRAFT` , `PUBLISHED` , `CANCELED` , `CLOSED`	Cycle de vie d'une annonce
`AccommodationType`	`HOUSE` , `ROOM` , `OTHER`	Type de logement
`ReportStatus`	`PENDING` , `RESOLVED` , `DISMISSED`	Signalement
`RentStatus`	`PAID` , `PARTIAL` , `LATE` , `UPCOMING`	Loyer d'un mois
`PaymentStatus`	`PENDING` , `SUCCESS` , `FAILED`	Paiement
`Operator`	`MTN` , `MOOV` , `CASH`	Moyen de paiement
`IdentityStatus`	`PENDING` , `APPROVED` , `REJECTED`	Vérification d'identité

Tables principales (clés étrangères en `_id`) :

Table	Champs
`users`	id, nom, prenoms, email (unique), telephone (unique), password, role, actif, identity_verified
`announcements`	id, owner_id, housing_id, title, type, rent, deposit_months, city, district, rooms, description, status, published_at
`announcement_photos`	id, announcement_id, url, position
`favorites`	user_id, announcement_id (clé unique sur les deux)

Table	Champs
`housings`	id, owner_id, title, type, rent, due_day, city, district, code (unique)
`rentals`	id, housing_id, tenant_id, status, started_at, ended_at
`rent_periods`	id, rental_id, period (AAAA-MM), total, carried_over, due_date
`payments`	id, rental_id, period, amount, operator, phone, status, reference, provider_ref, paid_at
`identity_requests`	id, owner_id, id_image_url, selfie_url, status, reason, reviewed_by
`reports`	id, announcement_id, reporter_id, reason, details, status

Le reste à payer d'un mois se calcule à partir des paiements `SUCCESS` ; il n'est pas stocké pour éviter les écarts.

Questions ouvertes

Ces points décident de la forme finale de l'API ; ils sont à trancher avec le cahier des charges avant d'écrire les contrôleurs.

- [ ] **Téléphone** : 10 chiffres commençant par `01`, sans `+229`, est une hypothèse de l'app. Le back doit-il stocker avec l'indicatif ?
- [ ] **Nom** : l'app envoie `nom` (dernier mot) et `prenoms` (le reste) découpés depuis un seul champ. Le back garde-t-il ce découpage ou veut-on deux champs dans le formulaire ?
- [ ] **Paiement** : KKiaPay ou un autre agrégateur ? Cela détermine le webhook et la façon de confirmer un paiement.
- [ ] **Location** : le propriétaire doit-il confirmer chaque locataire qui rejoint avec un code, ou l'accès est-il immédiat ?
- [ ] **Paiement partiel** : y a-t-il un montant minimum par paiement ?
- [ ] **Annonce** : un logement a-t-il une seule annonce à la fois, ou plusieurs (une par chambre) ?
- [ ] **Identité** : un propriétaire non vérifié peut-il publier (annonce sans l'étiquette « Vérifié ») ou non du tout ?
- [ ] **Images privées** : comment l'admin voit-il les pièces d'identité stockées en mode privé (URL signée courte renvoyée par le back) ?

- [ ] **Notifications** : rappels de loyer par SMS, e-mail ou notification dans l'app ? Pas d'API prévue pour l'instant.
- [ ] **Avis** : le flux d'avis sur les logements ou les propriétaires n'est pas couvert ici.